S.No: 5	Exp. Name: ***Node.js Event Logger Using Packages, Events, Timers, and File I/O***	**Date: 2026-09-23**

Aim:

In this experiment, you will build a Node.js event logging system that connects four core concepts into one working flow. The `server.js`, browser UI, and CSS file are already provided in the default code. Your only task is to implement two things inside `logger.js`.

Your Task

Implement the following inside `logger.js`:

- Create a class **LogEmitter** that extends the built-in `EventEmitter` class.
- Implement the **startLogging** function. It must do the following:
    - Create an instance of **LogEmitter**.
- Register a listener on the `"log"` event. Each time the event fires, the listener must build a log entry in this exact format and append it to `LOG_FILE` using `fs.appendFileSync`:
- `[<uuid>] <timestamp> – <message>\n`
- where `uuid` comes from `uuidv4()` and `timestamp` comes from `new Date().toISOString()`.Use `setInterval` to emit the `"log"` event every `2000` ms with the message `"Event fired"`.Use `clearInterval` to stop the interval after 5 ticks.Emit a `"done"` event on the emitter after the 5th tick.Return the **LogEmitter** instance.

Note: Click **Start** in the browser UI to trigger your implementation and observe the live dashboard. Click **Reset** to clear and run again.

Source Code:

my-app/logger.js

```
const EventEmitter = require("events");
const fs = require("fs");
const path = require("path");
const { v4: uuidv4 } = require("uuid");

const LOG_FILE = path.join(__dirname, "logs", "events.log");

if (!fs.existsSync(path.join(__dirname, "logs"))) {
  fs.mkdirSync(path.join(__dirname, "logs"));
}

class LogEmitter extends EventEmitter{}
function startLogging() {
        const emitter = new LogEmitter();
                emitter.on("log",(message)=>{
                fs.appendFileSync(
                        LOG_FILE, `[${uuidv4()}] ${new
Date().toISOString()} - ${message}\n`
                );
        });
        let count = 0;
        const interval = setInterval(()=>{
                count++;
                emitter.emit("log", "Event fired");
                if (count === 5) {
                        clearInterval(interval);
                        emitter.emit("done");
                }
        }, 2000);
        return emitter;
}
module.exports = { startLogging, LOG_FILE };
```

my-app/package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "Node.js Events, Timers, Packages and File I/O",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "uuid": "^9.0.0"
  }
}
```

my-app/public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Node.js Event Logger</title>
  <link rel="stylesheet" href="/style.css" />
</head>
<body>

  <div class="page-header">
    <h1>Node.js Event Logger</h1>
    <p class="subtitle">Monitor live event activity powered by
your Node.js implementation</p>
  </div>

  <div class="main">

    <div class="stat-grid">
      <div class="stat-card">
        <div class="stat-label">Status</div>
        <div class="stat-value" id="stat-status">
          <span class="badge idle">Idle</span>
        </div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Ticks Fired</div>
        <div class="stat-value big" id="stat-ticks">0 <span
class="of">/ 5</span></div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Last Event Time</div>
        <div class="stat-value mono" id="stat-time">—</div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Log File Size</div>
        <div class="stat-value mono" id="stat-size">0 bytes</div>
      </div>
    </div>

    <div class="card">
      <div class="card-title">Last Generated UUID</div>
      <div class="uuid-display" id="stat-uuid">—</div>
```

```
    </div>

    <div class="card">
      <div class="card-title">Event Log</div>
      <div class="empty-state" id="empty-state">
        <span class="empty-icon">○</span>
        <p>No events yet. Click Start to begin.</p>
      </div>
      <div class="log-feed" id="log-feed"></div>
    </div>

    <div class="btn-row">
      <button id="start-btn"
onclick="handleStart()">Start</button>
      <button id="reset-btn" class="btn-secondary"
onclick="handleReset()">Reset</button>
    </div>

  </div>

  <script>
    let pollInterval = null;
    let lastCount = 0;

    function setStatus(type, label) {
      document.getElementById("stat-status").innerHTML = '<span
class="badge ' + type + '">' + label + '</span>';
    }

    window.handleStart = async function () {
      document.getElementById("start-btn").disabled = true;
      document.getElementById("reset-btn").disabled = true;
      setStatus("running", "Running");
      try {
        const res = await fetch("/start", { method: "POST" });
        const data = await res.json();
        if (data.status === "error") {
          setStatus("error", "Error");
          document.getElementById("stat-uuid").textContent =
data.message;
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
          return;
        }
        document.getElementById("empty-state").style.display =
"none";
```

```
        lastCount = 0;
        pollInterval = setInterval(pollLogs, 600);
      } catch (err) {
        setStatus("error", "Error");
        document.getElementById("start-btn").disabled = false;
        document.getElementById("reset-btn").disabled = false;
      }
    };

    window.handleReset = async function () {
      clearInterval(pollInterval);
      await fetch("/reset", { method: "POST" });
      lastCount = 0;
      document.getElementById("stat-ticks").innerHTML = '0 <span
class="of">/ 5</span>';
      document.getElementById("stat-time").innerHTML = "—";
      document.getElementById("stat-size").textContent = "0
bytes";
      document.getElementById("stat-uuid").innerHTML = "—";
      document.getElementById("log-feed").innerHTML = "";
      document.getElementById("empty-state").style.display =
"flex";
      document.getElementById("start-btn").disabled = false;
      document.getElementById("reset-btn").disabled = false;
      setStatus("idle", "Idle");
    };

    async function pollLogs() {
      try {
        const res = await fetch("/logs");
        const data = await res.json();
        const lines = data.lines || [];

        if (lines.length !== lastCount) {
          lastCount = lines.length;
          document.getElementById("stat-ticks").innerHTML =
lines.length + ' <span class="of">/ 5</span>';
          document.getElementById("stat-size").textContent =
data.fileSize + " bytes";

          if (lines.length > 0) {
            const lastLine = lines[lines.length - 1];
            const uuidMatch = lastLine.match(/\[([^\]]+)\]/);
            const timeMatch = lastLine.match(/\]\s+([^\s]+)\s+/);
            if (uuidMatch) document.getElementById("stat-
uuid").textContent = uuidMatch[1];
```

```
            if (timeMatch) document.getElementById("stat-
time").textContent = timeMatch[1].replace("T", " ").replace("Z",
" UTC");
          }

          const feed = document.getElementById("log-feed");
          feed.innerHTML = lines.map(function(line, i) {
            const uuidMatch = line.match(/\[([^\]]+)\]/);
            const timeMatch = line.match(/\]\s+([^\s]+)\s+/);
            const msgMatch  = line.match(/\u2014\s+(.*)/);
            const uuid = uuidMatch ? uuidMatch[1] : "—";
            const time = timeMatch ? timeMatch[1].replace("T", "
").replace("Z", " UTC") : "—";
            const msg  = msgMatch  ? msgMatch[1] : line;
            return '<div class="log-row">'
              + '<span class="log-num">#' + (i + 1) + '</span>'
              + '<div class="log-details">'
              + '<span class="log-msg">' + msg + '</span>'
              + '<span class="log-meta">' + time + '</span>'
              + '<span class="log-uuid">' + uuid + '</span>'
              + '</div></div>';
          }).join("");
          feed.scrollTop = feed.scrollHeight;
        }

        if (!data.isLogging && lines.length >= 5) {
          clearInterval(pollInterval);
          setStatus("done", "Done");
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
        }

        if (!data.isLogging && lines.length === 0) {
          clearInterval(pollInterval);
          setStatus("error", "Error");
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
        }

      } catch (err) {
        clearInterval(pollInterval);
      }
    }
  </script>
```

```
</body>
</html>
```

my-app/public/style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: "Segoe UI", sans-serif;
  background: #f0f2f5;
  min-height: 100vh;
  color: #1a202c;
}

.page-header {
  background: #fff;
  border-bottom: 1px solid #e2e8f0;
  padding: 24px 40px;
  text-align: center;
}

.page-header h1 {
  font-size: 1.4rem;
  font-weight: 700;
  color: #1a202c;
  margin-bottom: 4px;
}

.subtitle { font-size: 0.83rem; color: #718096; }

.main {
  max-width: 680px;
  margin: 32px auto;
  padding: 0 24px;
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.stat-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 14px;
}

.stat-card {
  background: #fff;
  border: 1px solid #e2e8f0;
  border-radius: 12px;
```

```
  padding: 18px 20px;
}

.stat-label {
  font-size: 0.72rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.07em;
  color: #a0aec0;
  margin-bottom: 10px;
}

.stat-value { font-size: 0.95rem; font-weight: 600; color:
#2d3748; }

.stat-value.big {
  font-size: 1.8rem;
  font-weight: 700;
  color: #1a202c;
  line-height: 1;
}

.stat-value.mono {
  font-family: monospace;
  font-size: 0.82rem;
  color: #4a5568;
  word-break: break-all;
}

.of { font-size: 1rem; color: #a0aec0; font-weight: 500; }

.badge {
  display: inline-block;
  padding: 3px 10px;
  border-radius: 20px;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.badge.idle    { background: #edf2f7; color: #718096; }
.badge.running { background: #ebf8ff; color: #2b6cb0; border: 1px
solid #90cdf4; }
.badge.done    { background: #f0fff4; color: #276749; border: 1px
solid #9ae6b4; }
```

```
.badge.error   { background: #fff5f5; color: #c53030; border: 1px
solid #feb2b2; }

.card {
  background: #fff;
  border-radius: 12px;
  border: 1px solid #e2e8f0;
  padding: 20px 24px;
}

.card-title {
  font-size: 0.72rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.07em;
  color: #a0aec0;
  margin-bottom: 12px;
}

.uuid-display {
  font-family: monospace;
  font-size: 0.88rem;
  color: #3182ce;
  font-weight: 600;
  background: #ebf8ff;
  border-radius: 7px;
  padding: 10px 14px;
  word-break: break-all;
  letter-spacing: 0.03em;
}

.empty-state {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 10px;
  padding: 32px 20px;
  text-align: center;
}

.empty-icon { font-size: 2rem; color: #e2e8f0; line-height: 1; }
.empty-state p { font-size: 0.83rem; color: #a0aec0; }

.log-feed {
  display: flex;
```

```
  flex-direction: column;
  gap: 8px;
  max-height: 280px;
  overflow-y: auto;
}

.log-row {
  display: flex;
  align-items: flex-start;
  gap: 12px;
  background: #f7fafc;
  border-radius: 8px;
  padding: 10px 14px;
  border-left: 3px solid #4299e1;
}

.log-num {
  font-size: 0.72rem;
  font-weight: 700;
  color: #a0aec0;
  min-width: 24px;
  padding-top: 2px;
}

.log-details {
  display: flex;
  flex-direction: column;
  gap: 3px;
  flex: 1;
}

.log-msg { font-size: 0.88rem; font-weight: 600; color: #2d3748;
}
.log-meta { font-size: 0.75rem; font-family: monospace; color:
#718096; }
.log-uuid { font-size: 0.72rem; font-family: monospace; color:
#a0aec0; word-break: break-all; }

.btn-row { display: flex; gap: 12px; }

button {
  flex: 1;
  padding: 11px;
  background: #3182ce;
  color: #fff;
  border: none;
```

```
  border-radius: 8px;
  font-size: 0.9rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s, opacity 0.2s;
}

button:hover   { background: #2b6cb0; }
button:disabled { opacity: 0.5; cursor: not-allowed; }

.btn-secondary {
  background: #fff;
  color: #4a5568;
  border: 1.5px solid #e2e8f0;
}

.btn-secondary:hover { background: #f7fafc; }
```

my-app/server.js

```
const http = require("http");
const fs = require("fs");
const path = require("path");

const PORT = 3000;

const SERVER_FILE = path.join(__dirname, "logger.js");
let lastMtime = fs.statSync(SERVER_FILE).mtimeMs;
setInterval(() => {
  try {
    const mtime = fs.statSync(SERVER_FILE).mtimeMs;
    if (mtime !== lastMtime) {
      console.log("logger.js changed — restarting...");
      process.exit(1);
    }
  } catch (e) {}
}, 1000);

function serveStatic(res, filePath, contentType) {
  fs.readFile(filePath, (err, data) => {
    if (err) {
      res.writeHead(404, { "Content-Type": "text/plain" });
      res.end("404 Not Found");
      return;
    }
    res.writeHead(200, { "Content-Type": contentType });
    res.end(data);
  });
}

let isLogging = false;

function handleRequest(req, res) {
  if (req.method === "GET" && req.url === "/") {
    serveStatic(res, path.join(__dirname, "public",
"index.html"), "text/html");
    return;
  }

  if (req.method === "GET" && req.url === "/style.css") {
    serveStatic(res, path.join(__dirname, "public", "style.css"),
"text/css");
    return;
  }
```

```
  if (req.method === "POST" && req.url === "/start") {
    if (isLogging) {
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "already running" }));
      return;
    }
    try {
      delete require.cache[require.resolve("./logger")];
      const { startLogging, LOG_FILE } = require("./logger");
      if (typeof startLogging !== "function") {
        res.writeHead(500, { "Content-Type": "application/json"
});
        res.end(JSON.stringify({ status: "error", message:
"startLogging is not implemented yet." }));
        return;
      }
      const logDir = path.join(__dirname, "logs");
      if (!fs.existsSync(logDir)) fs.mkdirSync(logDir);
      fs.writeFileSync(LOG_FILE, "");
      isLogging = true;
      const emitter = startLogging();
      emitter.on("done", () => { isLogging = false; });
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "started" }));
    } catch (err) {
      isLogging = false;
      res.writeHead(500, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "error", message:
err.message }));
    }
    return;
  }

  if (req.method === "POST" && req.url === "/reset") {
    isLogging = false;
    try {
      delete require.cache[require.resolve("./logger")];
      const { LOG_FILE } = require("./logger");
      if (fs.existsSync(LOG_FILE)) fs.writeFileSync(LOG_FILE,
"");
    } catch (e) {}
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ status: "reset" }));
    return;
  }
```

```
  if (req.method === "GET" && req.url === "/logs") {
    try {
      delete require.cache[require.resolve("./logger")];
      const { LOG_FILE } = require("./logger");
      const content = fs.existsSync(LOG_FILE) ?
fs.readFileSync(LOG_FILE, "utf8") : "";
      const lines = content.trim().split("\n").filter(Boolean);
      const fileSize = fs.existsSync(LOG_FILE) ?
fs.statSync(LOG_FILE).size : 0;
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ lines, isLogging, fileSize }));
    } catch (err) {
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ lines: [], isLogging: false,
fileSize: 0 }));
    }
    return;
  }

  res.writeHead(404, { "Content-Type": "text/plain" });
  res.end("Not Found");
}

const server = http.createServer(handleRequest);
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```